Ligero 성능 특성 분석

개요

Ligero 프로토콜의 성능 특성은 **O(√N) 증명 크기**라는 핵심 장점과 함께 다양한 트레이드오프를 포함합니다. 이 문서에서는 쿼리 복잡도, 증명 크기, 메모리 사용량, 병렬화 가능성을 종합적으로 분석하고, 다른 프로토콜과의 비교를 통해 Ligero의 적용 시나리오를 제시합니다.

쿼리 복잡도와 증명 크기의 트레이드오프

1. 이론적 복잡도 분석

1.1 기본 복잡도 메트릭

| 메트릭 | Ligero | 설명 |

|--------|--------|------|

| **증명 크기** | O(√N) | N은 회로 크기 (제약 조건 수) |

| **증명 시간** | O(N log N) | FFT 연산과 Reed-Solomon 인코딩 포함 |

| **검증 시간** | O(√N) | 쿼리 수에 비례 |

| **쿼리 복잡도** | O(λ) | λ는 보안 매개변수 |

| **라운드 복잡도** | 2 | 공개 코인 프로토콜 |

1.2 증명 크기 구성 요소 분석

```
// BCS 변환 후 총 증명 크기

size_t total_proof_size = IOP_size + BCS_size;
```

```
// IOP 부분: 실제 필드 원소들
size_t IOP_size = prover_messages_size + query_responses_size;


// BCS 부분: 머클 트리 루트와 인증 경로
size_t BCS_size = MT_roots_size + MT_membership_proofs_size + pow_size;
```

구성 요소별 크기 분석:

1. **증명자 메시지 (Prover Messages)**

- 크기: O(√N) × field_size
- 내용: 직접 LDT를 위한 다항식 계수들
- 최적화: 계수 압축 및 배치 전송

2. **쿼리 응답 (Query Responses)**

- 크기: O(λ·√N) × field_size
- 내용: 무작위 위치에서의 오라클 값들
- 최적화: 쿼리 위치 최적화

3. **머클 트리 인증 경로**

- 크기: O(λ log N) × hash_size
- 내용: 쿼리된 위치의 인증 경로
- 최적화: 경로 가지치기 (pruning)

2. 실제 구현에서의 크기 최적화

2.1 매개변수 기반 크기 조정

```
// ligero_iop_parameters에서의 크기 영향 요소들
class ligero_iop_parameters {
float height_width_ratio_; // 행렬 형태 결정
size_t RS_extra_dimensions_; // Reed-Solomon 확장 비율
size_t num_query_phase_repetitions_; // 쿼리 반복 횟수
size_t num_interaction_phase_repetitions_; // 상호작용 반복 횟수
};
```

매개변수별 크기 영향:

매개변수	값 증가 시 영향	트레이드오프
`height_width_ratio`	쿼리 수 증가	인코딩 비용 vs 쿼리 비용
`RS_extra_dimensions`	인코딩 크기 지수적 증가	보안성 vs 효율성
`query_repetitions`	쿼리 응답 크기 선형 증가	보안성 vs 크기

2.2 동적 크기 최적화

```
// 영지식성 여부에 따른 동적 쿼리 수 계산
if (make_zk_) {
// 순환 의존성 해결을 위한 반복적 계산
size_t estimated_queries = 1;
while (estimated_queries != last_calculated) {
size_t query_bound = estimated_queries + 1;
size_t query_error_numerator = 2*H + 2*query_bound - 2;
```

```
// 수렴할 때까지 반복
}
} else {
// 비영지식: 단순한 계산
size_t query_error_numerator = 2*H - 2;
}
```

3. 보안 수준별 성능 특성

3.1 보안 매개변수와 증명 크기

```
보안 수준 λ = 128인 경우:
- 쿼리 반복 횟수: ~40-60회
- 상호작용 반복 횟수: ~6-10회
- 총 증명 크기: ~100-500KB (회로 크기에 따라)

보안 수준 λ = 256인 경우:
- 쿼리 반복 횟수: ~80-120회
- 상호작용 반복 횟수: ~12-20회
- 총 증명 크기: ~200-1MB (회로 크기에 따라)
```

3.2 회로 크기별 확장성

```
// 회로 크기에 따른 성능 예측 (의사코드)
```

```
struct PerformanceMetrics {
size_t circuit_size;
size_t proof_size_bytes;
double proving_time_ms;
double verification_time_ms;
};

// 실제 측정 데이터 기반 추정
PerformanceMetrics estimate_performance(size_t  num_constraints) {
size_t sqrt_n = (size_t)sqrt(num_constraints);
return {
.circuit_size = num_constraints,
.proof_size_bytes = sqrt_n * 32 * 50, // 대략적 추정
.proving_time_ms = num_constraints * log2(num_constraints) * 0.01,
.verification_time_ms = sqrt_n * 0.1
};
}
```

메모리 사용량과 병렬화 가능성 분석

1. 메모리 사용 패턴

1.1 증명자 메모리 사용량

```
// 주요 메모리 사용 구성 요소
struct LigeroMemoryUsage {
// 원본 증인 행렬
size_t witness_matrix_size = matrix_height * matrix_width * sizeof(FieldT);
// Reed-Solomon 인코딩된 행렬
size_t encoded_matrix_size = matrix_height * (matrix_width << RS_extra_dimension
// 머클 트리 저장
size_t merkle_tree_size = encoded_matrix_size * 2; // 내부 노드 포함
// FFT 작업 공간
size_t fft_workspace = matrix_width * sizeof(FieldT) * 2;
// 총 메모리 사용량
size_t total_memory = witness_matrix_size + encoded_matrix_size +
merkle_tree_size + fft_workspace;
};
```

메모리 최적화 전략:

1. **스트리밍 처리**: 행별로 순차 처리하여 메모리 사용량 감소
2. **인플레이스 연산**: FFT와 인코딩을 동일 메모리 공간에서 수행
3. **지연 계산**: 필요한 시점에만 머클 트리 노드 계산

1.2 검증자 메모리 사용량

```
// 검증자의 메모리 사용량 (증명자 대비 매우 적음)
struct VerifierMemoryUsage {
```

```
// 쿼리 응답 저장
size_t query_responses = num_queries * query_size * sizeof(FieldT);
// 머클 트리 인증 경로
size_t auth_paths = num_queries * log2(domain_size) * hash_size;
// 작업 공간
size_t workspace = query_size * sizeof(FieldT);
// 총 메모리 (일반적으로 < 1MB)
size_t total_memory = query_responses + auth_paths + workspace;
};
```

2. 병렬화 가능성과 최적화

2.1 증명 생성 단계별 병렬화

```
// 1단계: 행별 Reed-Solomon 인코딩 (완전 병렬화 가능)
void parallel_rs_encoding(const Matrix& witness_matrix) {
#pragma omp parallel for
for (size_t row = 0; row < matrix_height; ++row) {
encode_row_with_rs(witness_matrix[row]);
}
}

// 2단계: 머클 트리 구성 (부분 병렬화 가능)
void parallel_merkle_tree_construction(const EncodedMatrix& encoded) {
```

```
// 리프 레벨: 완전 병렬화
#pragma omp parallel for
for (size_t i = 0; i < num_leaves; ++i) {
compute_leaf_hash(encoded[i]);
}
// 내부 노드: 레벨별 병렬화
for (size_t level = 0; level < tree_height; ++level) {
#pragma omp parallel for
for (size_t node = 0; node < nodes_at_level[level]; ++node) {
compute_internal_node(level, node);
}
}
}

// 3단계: 쿼리 응답 생성 (완전 병렬화 가능)
void parallel_query_response(const std::vector<QueryPosition>& queries) {
#pragma omp parallel for
for (size_t i = 0; i < queries.size(); ++i) {
generate_query_response(queries[i]);
}
}
```

2.2 SIMD 최적화 기회

```
// 벡터화 가능한 연산들
class SIMDOptimizations {
public:
// 필드 원소 배치 연산
void vectorized_field_operations(const FieldT* a, const FieldT* b, FieldT* r
// AVX2/AVX-512를 사용한 병렬 필드 연산
for (size_t i = 0; i < count; i += SIMD_WIDTH) {
// 벡터 로드, 연산, 저장
}
}
// 해시 함수 배치 처리
void vectorized_hashing(const std::vector<std::vector<FieldT>>& inputs) {
// Blake2b의 병렬 구현 활용
// 여러 해시를 동시에 계산
}
// FFT 벡터화
void vectorized_fft(FieldT* coefficients, size_t size) {
// FFTW 또는 커스텀 벡터화된 FFT 사용
}
};
```

2.3 GPU 가속 가능성

```
// GPU 가속이 효과적인 부분들
struct GPUAcceleration {
// 1. 대규모 병렬 FFT
void gpu_fft_batch(const std::vector<std::vector<FieldT>>& polynomials) {
// CUDA FFT 라이브러리 활용
// 수백-수천 개의 다항식을 동시에 처리
}
// 2. Reed-Solomon 인코딩
void gpu_rs_encoding(const Matrix& witness_matrix) {
// 각 행을 GPU 스레드에서 병렬 처리
// 메모리 대역폭 최적화
}
// 3. 해시 트리 구성
void gpu_merkle_tree(const EncodedMatrix& leaves) {
// 트리 레벨별 병렬 해시 계산
// 공유 메모리 활용으로 지연시간 감소
}
};
```

3. 실제 성능 측정 결과

3.1 벤치마크 환경 설정

```
// instrument_ligero_snark.cpp에서의 성능 측정
```

```
template<typename FieldT, typename hash_type>
void  benchmark_ligero_performance() {
// 다양한 회로 크기에 대한 측정
for (size_t log_n = 10; log_n <= 20; ++log_n) {
size_t n = 1ul << log_n;
// R1CS 예제 생성
r1cs_example<FieldT> example = generate_r1cs_example<FieldT>(n, 15, n-1);
// 증명 생성 시간 측정
auto start = std::chrono::high_resolution_clock::now();
auto proof = ligero_snark_prover(example.constraint_system_,
example.primary_input_,
example.auxiliary_input_,
parameters);
auto end = std::chrono::high_resolution_clock::now();
// 결과 출력
printf("Circuit size: %zu, Proof size: %zu bytes, Time: %f ms\n",
n, proof.size_in_bytes(),
std::chrono::duration<double, std::milli>(end - start).count());
}
}
```

3.2 실측 성능 데이터 (추정)

| 회로 크기 | 증명 크기 | 증명 시간 | 검증 시간 | 메모리 사용량 |

|----------|----------|----------|----------|----------|

| 2^10 (1K) | ~50 KB | ~10 ms | ~1 ms | ~10 MB |

| 2^15 (32K) | ~180 KB | ~200 ms | ~5 ms | ~100 MB |

| 2^20 (1M) | ~1 MB | ~5 sec | ~20 ms | ~1 GB |

| 2^25 (32M) | ~5.6 MB | ~3 min | ~100 ms | ~10 GB |

다른 프로토콜 대비 장단점과 적용 시나리오

1. 주요 프로토콜과의 비교

1.1 복잡도 비교표

| 프로토콜 | 증명 크기 | 증명 시간 | 검증 시간 | 신뢰 설정 | 양자 후 보안 |

|----------|----------|----------|----------|----------|----------|

| **Ligero** | O(√N) | O(N log N) | O(√N) | 없음 | 있음 |

| **Aurora** | O($\log^2$ N) | O(N log N) | O($\log^2$ N) | 없음 | 있음 |

| **Fractal** | O(log N) | O(N log N) | O(log N) | 있음 | 있음 |

| **Groth16** | O(1) | O(N log N) | O(1) | 있음 | 없음 |

| **PLONK** | O(1) | O(N log N) | O(1) | 범용 | 없음 |

1.2 실용적 특성 비교

```
// 각 프로토콜의 실용적 고려사항
struct ProtocolCharacteristics {
string name;
bool transparent; // 투명성 (신뢰 설정 불필요)
```

```
bool post_quantum; // 양자 후 보안성

bool universal_setup; // 범용 설정 가능

size_t setup_complexity; // 설정 복잡도

size_t implementation_complexity; // 구현 복잡도

};


ProtocolCharacteristics protocols[] = {

{"Ligero", true, true, true, 1, 3},

{"Aurora", true, true, true, 1, 4},

{"Fractal", false, true, true, 3, 5},

{"Groth16", false, false, false, 5, 2},

{"PLONK", false, false, true, 4, 3}

};
```

2. Ligero의 고유 장점

2.1 투명성과 단순성

```
// Ligero의 투명성 구현

class LigeroTransparency {

public:

// 1. 신뢰할 수 있는 설정 불필요

static bool requires_trusted_setup() { return false; }

// 2. 공개적으로 검증 가능한 무작위성
```

```
static bool uses_public_randomness() { return true; }

// 3. 단순한 암호학적 가정

static std::vector<string> cryptographic_assumptions() {

return {"해시 함수의 충돌 저항성", "Reed-Solomon 코드의 거리 특성"};

}

// 4. 구현 단순성

static size_t implementation_lines_of_code() { return 5000; } // 추정

};
```

2.2 양자 후 보안성

```
// 양자 컴퓨터에 대한 저항성

class PostQuantumSecurity {

public:

// 해시 기반 보안 (양자 저항성)

static bool hash_based_security() { return true; }

// 대칭키 암호학 기반

static bool symmetric_crypto_only() { return true; }

// 이산 로그 문제 의존성 없음

static bool no_discrete_log_assumption() { return true; }

// 격자 문제 의존성 없음

static bool no_lattice_assumption() { return true; }

};
```

3. 적용 시나리오 분석

3.1 최적 적용 분야

```
// Ligero가 최적인 시나리오들
enum class OptimalScenario {
// 1. 중간 규모 회로 (10K - 1M 제약)
MEDIUM_SCALE_CIRCUITS,
// 2. 투명성이 중요한 애플리케이션
TRANSPARENCY_CRITICAL,
// 3. 양자 후 보안이 필요한 시스템
POST_QUANTUM_REQUIRED,
// 4. 빠른 프로토타이핑이 필요한 경우
RAPID_PROTOTYPING,
// 5. 교육 및 연구 목적
EDUCATIONAL_RESEARCH
};

class ScenarioAnalysis {
public:
static bool is_optimal_for(OptimalScenario scenario, size_t circuit_size) {
switch (scenario) {
case MEDIUM_SCALE_CIRCUITS:
return circuit_size >= 1000 && circuit_size <= 1000000;
```

```
case TRANSPARENCY_CRITICAL:
return true; // 항상 최적
case POST_QUANTUM_REQUIRED:
return true; // 항상 최적
case RAPID_PROTOTYPING:
return true; // 구현 단순성으로 인해
case EDUCATIONAL_RESEARCH:
return true; // 이해하기 쉬운 구조
}
return false;
}
};
```

3.2 비최적 시나리오

```
// Ligero가 비최적인 경우들
enum  class SuboptimalScenario {
// 1. 매우 큰 회로 (> 10M 제약)
VERY_LARGE_CIRCUITS,
// 2. 증명 크기가 중요한 애플리케이션
PROOF_SIZE_CRITICAL,
// 3. 네트워크 대역폭이 제한적인 환경
BANDWIDTH_LIMITED,
// 4. 빈번한 검증이 필요한 시스템
```

```
FREQUENT_VERIFICATION,
// 5. 모바일/IoT 환경
RESOURCE_CONSTRAINED
};


class SuboptimalAnalysis {
public:
static string get_better_alternative(SuboptimalScenario scenario) {
switch (scenario) {
case VERY_LARGE_CIRCUITS:
return  "Aurora (O(log² N) 증명 크기)";
case PROOF_SIZE_CRITICAL:
return  "Fractal (O(log N) 증명 크기)";
case BANDWIDTH_LIMITED:
return  "Groth16 (상수 크기 증명)";
case FREQUENT_VERIFICATION:
return  "PLONK (빠른 검증)";
case RESOURCE_CONSTRAINED:
return  "경량화된 Bulletproofs";
}
return  "상황에 따라 다름";
}
```

```
};
```

4. 실제 배포 고려사항

4.1 하드웨어 요구사항

```
// 권장 하드웨어 사양
struct HardwareRequirements {
// CPU: 멀티코어 프로세서 (병렬화 활용)
size_t recommended_cores = 8;
// 메모리: 회로 크기의 10-100배
size_t memory_multiplier = 50;
// 저장공간: 임시 파일용
size_t storage_gb = 10;
// 네트워크: 증명 전송용
size_t bandwidth_mbps = 100;
static HardwareRequirements for_circuit_size(size_t  num_constraints) {
HardwareRequirements req;
req.memory_multiplier = std::max(10ul, 100000000ul / num_constraints);
return req;
}
};
```

4.2 소프트웨어 최적화 권장사항

```
// 배포 시 최적화 체크리스트

class DeploymentOptimizations {

public:

static std::vector<string> get_optimization_checklist() {

return {

"컴파일러 최적화 플래그 활성화 (-O3, -march=native)",

"OpenMP 병렬화 활성화",

"SIMD 명령어 활용 (AVX2/AVX-512)",

"메모리 풀링으로 할당 오버헤드 감소",

"FFT 라이브러리 최적화 (FFTW 등)",

"해시 함수 하드웨어 가속 활용",

"쿼리 배치 처리로 캐시 효율성 향상",

"증명 압축 및 직렬화 최적화"

};

}

};
```

이러한 종합적인 성능 분석을 통해 Ligero는 투명성과 양자 후 보안성이 중요한 중간 규모 애플리케이션에서 최적의 선택임을 확인할 수 있습니다.